

Détection d'anomalies par Deep Learning

Auto-encodeur convolutif, exploration comparative, PatchCore et choix final de production — MVTec AD

Synthèse de l'ensemble des travaux menés dans le cadre du TP « B6 partie 2 — Auto-encodeur & détection d'anomalies » : comparaison de frameworks et de matériels, effets de la résolution et de la taille de noyau de convolution, politiques de seuil, réalisation du TP demandé par la consigne (conception, entraînement, score, heatmaps), quatre pistes d'amélioration testées empiriquement — dont une confrontation à PatchCore (bibliothèque *anomalib*) — puis la traduction d'une décision métier (maximiser la détection) en un modèle et un seuil de production, désormais implémentés dans `src/indusense/vision/patchcore.py`.

Formation	Concevoir et implémenter une solution d'IA — parcours IT
Module	B6 partie 2 — Deep Learning / détection d'anomalies (modules 15 à 18)
Jeu de données	MVTec AD — catégories « wood » (texture) et « metal_nut » (objet)
Matériel GPU	AMD Radeon RX 9070 XT, ROCm 7.2.1, PyTorch 2.9.1
Notebooks couverts	02a, 02b, 02c (+ variantes 512px et noyau 5×5), 02d, 02e, 02f, tp_deep_learning, 03-choix-final
Décision métier	« Mieux vaut provoquer trop d'inspections que de laisser passer trop de défauts »
Modèle retenu pour la production	PatchCore (wide_resnet50_2), seuil au 99e centile — implémenté dans <code>src/indusense/vision/patchcore.py</code>

0. Contexte et protocole commun

Objectif métier : détecter des défauts sur des pièces industrielles (planches de bois, écrous métalliques) sans disposer d'images de défauts annotées pour l'entraînement — seules des pièces **saines** sont disponibles. L'approche retenue est un **auto-encodeur convolutif** entraîné à reconstruire uniquement des images saines : l'**erreur de reconstruction** (MSE) sert ensuite de score d'anomalie sur des images inédites, saines ou défectueuses.

Architecture commune à toutes les expériences (sauf mention contraire) : encodeur 3 convolutions « stride 2 » (canaux 3→64→128→256), goulot d'étranglement, décodeur symétrique par déconvolutions, sortie *Sigmoid*. Images **256×256×3**, lot de 32, **60 époques**, optimiseur Adam ($lr=1e-3$), perte MSE. Seuil de décision calibré uniquement sur une **validation saine** (jamais vue à l'entraînement) ; qualité mesurée par ROC-AUC/PR-AUC (indépendants du seuil) puis précision/rappel/F1 au seuil choisi.

Chaque section ci-dessous suit le même plan : **Méthode** (ce qui change par rapport au protocole commun), **Justification** (pourquoi cette expérience), **Résultats** (chiffres mesurés et lecture).

1. TensorFlow vs PyTorch, CPU vs GPU

Méthode

Même architecture (741 379 paramètres, vérifié identique dans les deux frameworks) entraînée quatre fois : TensorFlow/Keras sur CPU, PyTorch sur CPU, PyTorch sur GPU (ROCm). Temps mesuré en isolant la **première époque** (coût d'initialisation — traçage *tf.function* côté Keras, compilation des noyaux HIP côté PyTorch+ROCm) de la **vitesse de croisière**.

Justification

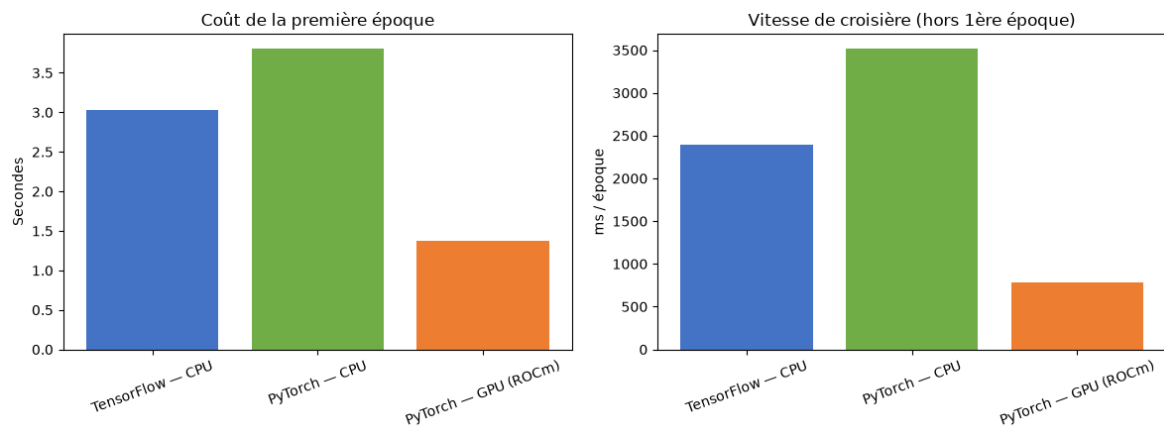
Choisir un framework et un matériel d'entraînement pour la suite du TP est une décision qui doit s'appuyer sur une mesure, pas une supposition — et vérifier qu'aucune perte de qualité ne compense un gain de vitesse.

Résultats — temps d'entraînement

Exécution	Paramètres	1ère époque (s)	Croisière (ms/époque)	Total (s)
TensorFlow — CPU	741 379	3,03	2 390,1	144,05
PyTorch — CPU	741 379	3,80	3 512,5	211,04
PyTorch — GPU (ROCm)	741 379	1,38	786,4	47,78

Résultats — qualité de détection

Exécution	Seuil	ROC-AUC	PR-AUC	Précision	Rappel	F1
TensorFlow — CPU	0,00132	0,9605	0,9877	1,00	0,117	0,209
PyTorch — CPU	0,00106	0,9561	0,9861	1,00	0,100	0,182
PyTorch — GPU	0,00101	0,9632	0,9884	1,00	0,117	0,209



Coût de première époque et vitesse de croisière — TensorFlow CPU, PyTorch CPU, PyTorch GPU.

Constat. Le GPU accélère l'entraînement d'un facteur **4,47×** en régime de croisière, sans dégrader la qualité de détection. **Découverte méthodologique** obtenue en ré-exécutant les notebooks à plusieurs jours d'intervalle : **PyTorch est bit-reproductible** à seed fixée (poids, pertes et métriques strictement identiques d'une exécution à l'autre) alors que **TensorFlow/Keras sur CPU ne l'est pas** (les opérations parallèles oneDNN introduisent une non-déterminisme) — les chiffres TensorFlow ci-dessus ont varié entre deux exécutions du même notebook (ROC-AUC 0,9825 puis 0,9605), jamais les chiffres PyTorch.

2. Effet de la résolution d'image (256×256 vs 512×512)

Méthode

Même architecture et hyperparamètres que la référence GPU, résolution portée de 256×256 à 512×512 (goulot d'étranglement spatial 64×64 au lieu de 32×32, même profondeur de canaux).

Justification

Vérifier si une image plus grande, donc plus riche en détails de texture, améliore la détection — et quantifier le coût GPU réel d'un tel changement plutôt que de l'estimer a priori.

Résultats

Résolution	1ère époque (s)	Croisière (ms/époque)	Total (s)	ROC-AUC	PR-AUC
256×256	1,38	786,4	47,78	0,9632	0,9884
512×512	3,97	3 219,6	193,92	0,9482	0,9835

Constat. Une fois la forme 512×512 déjà connue de MIOpen (bibliothèque de noyaux ROCm), croisière et première époque suivent toutes deux le facteur ~4 attendu (4× plus de pixels), sans gain de qualité — 256×256 offre ici un meilleur rapport temps/qualité. **Découverte méthodologique :** lors de la *toute première* exécution de cette forme sur cette machine, la première époque avait explosé à 71,9 s (facteur 52×, pas 4×) — un coût d'auto-tuning MIOpen **punctuel et mis en cache sur disque**, qui ne réapparaît pas aux exécutions suivantes. Un facteur mesuré une seule fois peut donc induire en erreur si on le prend pour une propriété stable.

3. Effet de la taille du noyau de convolution (3×3 vs 5×5)

Méthode

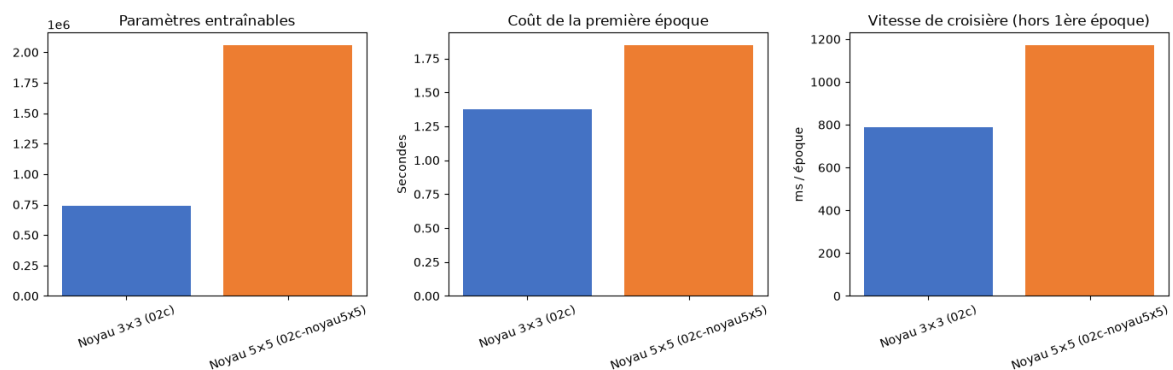
Résolution, hyperparamètres et device identiques à la référence GPU ; seule la taille des noyaux de convolution change (padding et *output_padding* recalculés pour conserver un goulot d'étranglement spatial identique 32×32).

Justification

Tester si un champ réceptif plus large (donc plus de paramètres) améliore la reconstruction ou la détection, et mesurer le coût réel en temps et en paramètres d'un tel changement.

Résultats

Noyau	Paramètres	Croisière (ms/ép.)	Total (s)	ROC-AUC	PR-AUC	Rappel @ seuil
3×3	741 379	786,4	47,78	0,9632	0,9884	0,117
5×5	2 058 243	1 171,7	70,98	0,9535	0,9858	0,033



Paramètres, coût de première époque et vitesse de croisière — noyaux 3×3 vs 5×5.

Constat. Les paramètres suivent exactement le ratio théorique des surfaces de noyau $((5 \times 5)/(3 \times 3) = 25/9 \approx 2,78\times)$, mais la vitesse de croisière n'augmente que de $1,49\times$: l'entraînement n'est pas purement limité par le volume de calcul à cette échelle. Le classement se dégrade légèrement et le rappel au seuil moyenne + 3σ chute (0,033 contre 0,117) — non pas par perte de capacité de détection (les ratios erreur défaut / erreur saine restent quasi identiques entre les deux noyaux) mais parce que la validation saine du noyau 5×5 est **proportionnellement plus dispersée** (écart-type / moyenne = 0,411 contre 0,357), ce qui pousse un seuil moyenne + n-écart-type plus loin dans la distribution des erreurs.

4. Comparaison de politiques de seuil (autour du seuil par défaut)

Méthode

Reprise du modèle de référence GPU (poids, ROC-AUC/PR-AUC identiques). Sept politiques de seuil calibrées **uniquement sur la validation saine** (37 images) : moyenne + $n \cdot \text{écart-type}$ pour $n \in \{1, 2, 3\}$ et percentile $\in \{90, 95, 97, 99\}$.

Justification

Le seuil moyenne + 3σ , utilisé par défaut, s'était révélé très conservateur (rappel $\approx 0,12$) malgré un excellent ROC-AUC — la politique de décision méritait d'être questionnée indépendamment de la qualité du classement.

Résultats

Politique	Fausse alertes (validation)	Rappel réel (test)
moyenne + 1σ	24,3 %	0,35
moyenne + 2σ	2,7 %	0,18
moyenne + 3σ	0,0 %	0,12
percentile 90	10,8 %	0,25
percentile 95	5,4 %	0,20
percentile 97	5,4 %	0,18
percentile 99	2,7 %	0,18

Constat. La précision reste à 1,00 pour les 7 politiques (aucune fausse alerte sur les 19 images saines de test, même la plus permissive) : le taux de fausses alertes estimé sur 37 images de validation ne s'est pas confirmé sur les 19 images de test — un effet d'échantillonnage sur de petits jeux de données. Cette première exploration (seuils resserrés autour de la politique par défaut) sera reprise en section 7 avec un balayage bien plus large, une fois la décision métier de maximiser le rappel explicitée.

5. Le TP demandé : conception, entraînement, seuil, heatmaps

Notebook `tp_deep_learning.ipynb`, conforme à la consigne `b6_deep_learning_auto_encodeur.md` : même architecture que ci-dessus, entraînement suivi dans **MLflow** (store local SQLite), seuil calibré au **99e centile** de la validation (plutôt que moyenne + 3σ), et deux évaluations quantitatives nouvelles : l'**AUROC pixel** (localisation, via les masques de vérité terrain) en complément de l'AUROC image (détection).

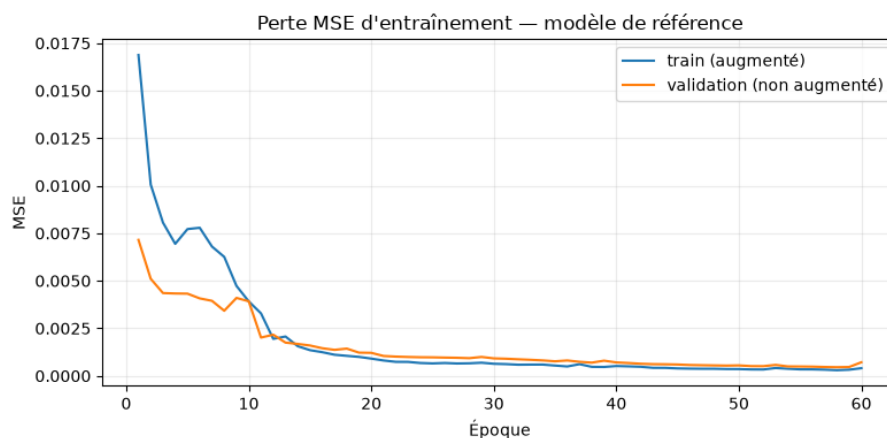
5.1 — Conception : le ratio de compression

Méthode. Avant tout entraînement, calcul du ratio (valeurs en entrée) $\div$ (valeurs dans le goulot d'étranglement).

Justification. La consigne prévient qu'un ratio proche de 1 (ou inférieur) autorise le modèle à apprendre une quasi-identité, détruisant le signal d'anomalie — un diagnostic disponible avant même d'entraîner.

Résultat. Goulot $32 \times 32 \times 256 = 262\,144$ valeurs, contre 196 608 valeurs en entrée ($256 \times 256 \times 3$) : ratio = **0,75**, inférieur à 1 — le goulot est en réalité *plus grand* que l'entrée. Le piège annoncé par la consigne est vérifié numériquement avant le premier entraînement.

5.2 — Entraînement

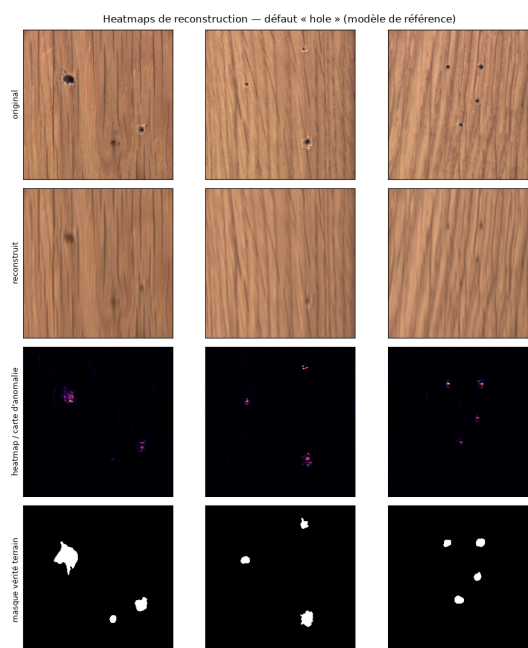


Perte MSE (train augmenté vs validation non augmentée) — modèle de référence du TP, 60 époques.

Les deux courbes restent proches et décroissent ensemble jusqu'à un plateau bas — aucun sur-apprentissage marqué, cohérent avec un modèle largement assez capable de reconstruire les images saines (ratio de compression < 1).

5.3 — Score, seuil et évaluation

Métrique	Valeur
Seuil (validation, 99e centile)	0,001041
ROC-AUC image	0,9158
PR-AUC image	0,9705
AUROC pixel (via masques)	0,6375
Précision @ seuil	1,00
Rappel @ seuil	0,217 (13 / 60)
F1 @ seuil	0,356



Heatmaps — défaut « hole » (wood) : localisation nette, bien alignée avec le masque.

Analyse critique (livrable attendu). Ce qui marche : les défauts nets et localisés (hole) produisent un point chaud net, bien positionné par rapport au masque de vérité terrain. Ce qui échoue : le rappel reste modeste

(0,217) et les défauts diffus (liquid) sont presque aussi bien reconstruits que les images saines — la heatmap n'y montre qu'un bruit diffus. Pourquoi : le ratio de compression < 1 (5.1) donne au modèle une large capacité à reconstruire fidèlement des motifs jamais vus à l'entraînement, y compris des altérations discrètes — exactement le piège annoncé par la consigne, vérifié empiriquement.

6. Pour aller plus loin

Quatre pistes d'amélioration citées par la consigne, toutes testées empiriquement (et non simplement discutées) dans la même session.

6.1 — Resserer le goulot d'étranglement

Méthode. base_channels réduit de 64 à 16 (goulot 32×32×64, ratio de compression porté à 3,0 — une vraie compression). **Justification.** Tester directement l'hypothèse du piège d'identité de la section 5.1 : un goulot réellement compressif limite-t-il ce risque ?

Modèle	Paramètres	Ratio compr.	AUROC image	AUROC pixel	Rappel @ seuil
Référence (bc=64)	741 379	0,75	0,9158	0,6375	0,217
Goulot resserré (bc=16)	47 107	3,00	0,9202	0,6544	0,100

Résultat. Contrairement à la crainte d'un goulot trop large, forcer une vraie compression **améliore légèrement** le classement (AUROC image et pixel en hausse) avec 15,7× moins de paramètres. Le rappel au seuil retenu baisse, mais par un effet de **politique de seuil** (distribution de validation différente), pas par perte de capacité de détection.

6.2 — Perte SSIM au lieu de MSE

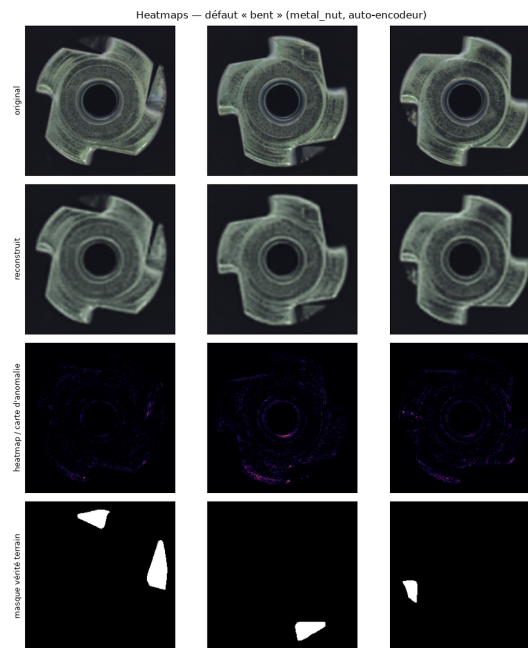
Méthode. Architecture identique à la référence, perte d'entraînement 1 – SSIM (comparaison par fenêtre glissante gaussienne : luminance, contraste, structure locale) au lieu de la MSE ; score d'anomalie final laissé en MSE pour rester comparable. **Justification.** La consigne suggère SSIM comme plus sensible aux altérations de texture qu'une simple différence pixel à pixel.

Modèle	AUROC image	AUROC pixel	Rappel @ seuil
Référence (perte MSE)	0,9158	0,6375	0,217
Perte SSIM	0,8579	0,7651	0,200

Résultat. Compromis net : l'AUROC **image** se dégrade mais l'AUROC **pixel** devient le meilleur des trois modèles wood testés. SSIM aide la *localisation*, pas nécessairement la *détection* globale telle que mesurée ici.

6.3 — Objet (metal_nut) vs texture (wood)

Méthode. Même auto-encodeur (MSE, bc=64) entraîné sur la catégorie MVTec AD metal_nut (pièce métallique) plutôt que wood (texture). Point de méthode : le pipeline d'augmentation de metal_nut exclut tout retournement (*flip*), car « **flip** » est un **défaut réel** de ce jeu de données (l'écrou retourné) — l'augmenter par retournement aurait détruit ce signal. **Justification.** Vérifier si l'hypothèse de l'auto-encodeur par reconstruction (erreur élevée = anomalie) tient aussi bien sur un objet que sur une texture.



Heatmaps — défaut « bent » (metal_nut) : le point chaud suit les reflets spéculaires, pas la déformation.

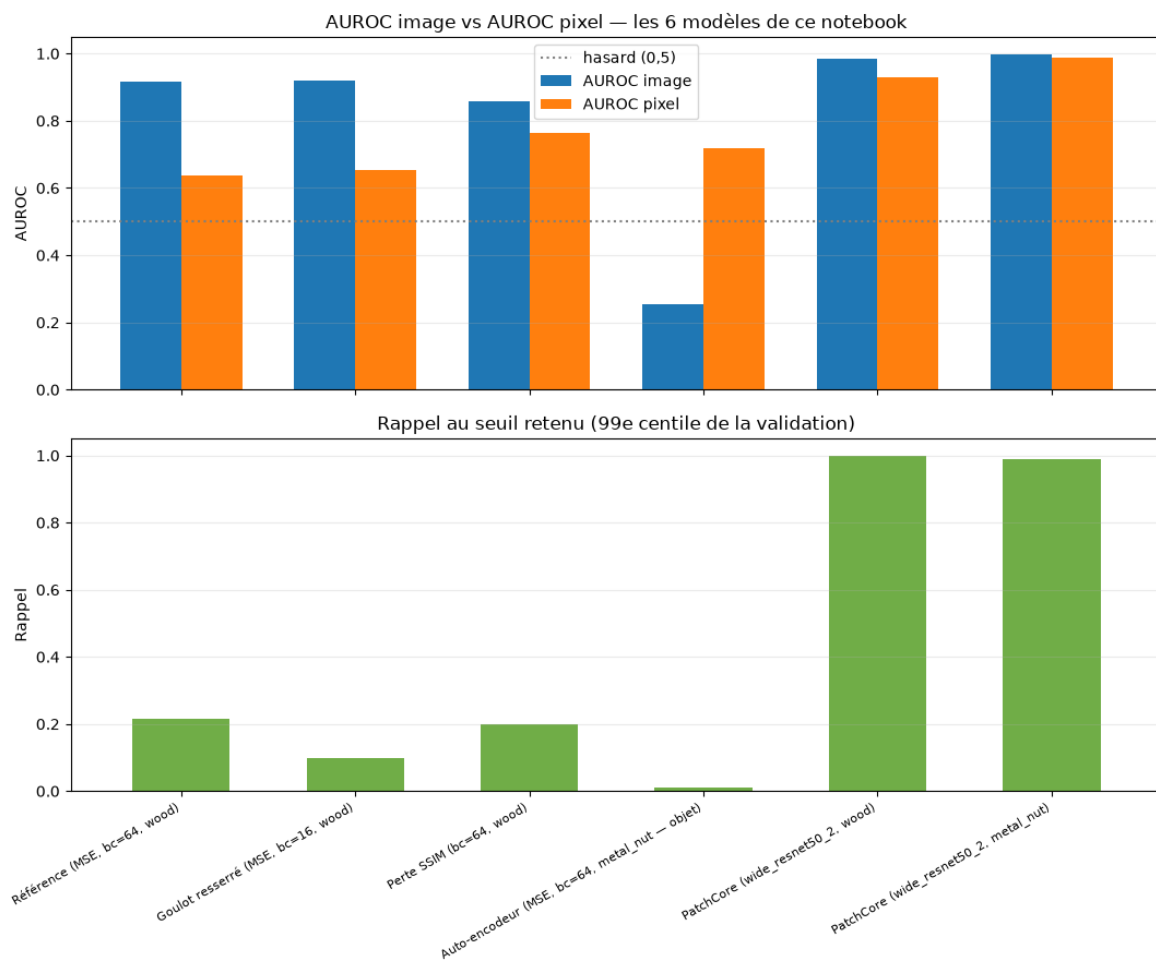
Résultat — échec quantifié. AUROC image = **0,2556**, inférieur à 0,5 : le score classe en moyenne les images saines comme plus suspectes que les défauts. Cause visible sur les heatmaps : les contours métalliques réfléchissants produisent un bruit de reconstruction présent sur toutes les images (saines ou non), qui l'emporte parfois sur le signal du défaut lui-même — un pan tordu (bent) est même lissé et reconstruit comme normal. L'hypothèse de l'auto-encodeur par reconstruction, qui tient sur une texture homogène, **casse sur un objet réfléchissant**.

6.4 — Confrontation à PatchCore (bibliothèque anomalib)

Méthode. PatchCore (Roth et al., 2022) : aucun entraînement par rétropropagation. Extraction de features d'un réseau wide_resnet50_2 pré-entraîné ImageNet sur les images saines, stockées dans une mémoire de patches (coreset 10 %), puis score de chaque patch de test par distance au plus proche voisin. **Justification.** Confronter le baseline pédagogique de ce TP à une méthode état de l'art citée par la consigne, en particulier sur le cas d'échec de la section 6.3.

Catégorie	Méthode	AUROC image	AUROC pixel	Rappel @ seuil	Précision @ seuil
wood (texture)	Auto-encodeur	0,9158	0,6375	0,217	1,00
wood (texture)	PatchCore	0,9860	0,9304	1,000	0,882
metal_nut (objet)	Auto-encodeur	0,2556	0,7175	0,011	0,333
metal_nut (objet)	PatchCore	0,9985	0,9866	0,989	0,979

Résultat. PatchCore surclasse l'auto-encodeur sur les deux catégories et **résout précisément l'échec** de la section 6.3 (AUROC image 0,9985 sur metal_nut, quasi insensible au changement de catégorie contrairement à l'auto-encodeur). Raison méthodologique : PatchCore compare des features déjà robustes aux variations photométriques (pré-entraînement ImageNet) par correspondance locale au plus proche voisin, jamais une image entière pixel à pixel — un reflet spéculaire inhabituel mais déjà présent dans la mémoire de patches saines ne perturbe donc pas le score.



Synthèse comparative des 6 modèles testés : AUROC image/pixel et rappel au seuil retenu.

7. Décision finale : maximiser la détection sur wood

Décision métier qui motive cette section : « *mieux vaut provoquer trop d'inspections que de laisser passer trop de défauts* » — le coût d'un défaut manqué est jugé bien supérieur au coût d'une inspection inutile. Notebook associé : 03-choix-final-detection-maximale-wood.ipynb.

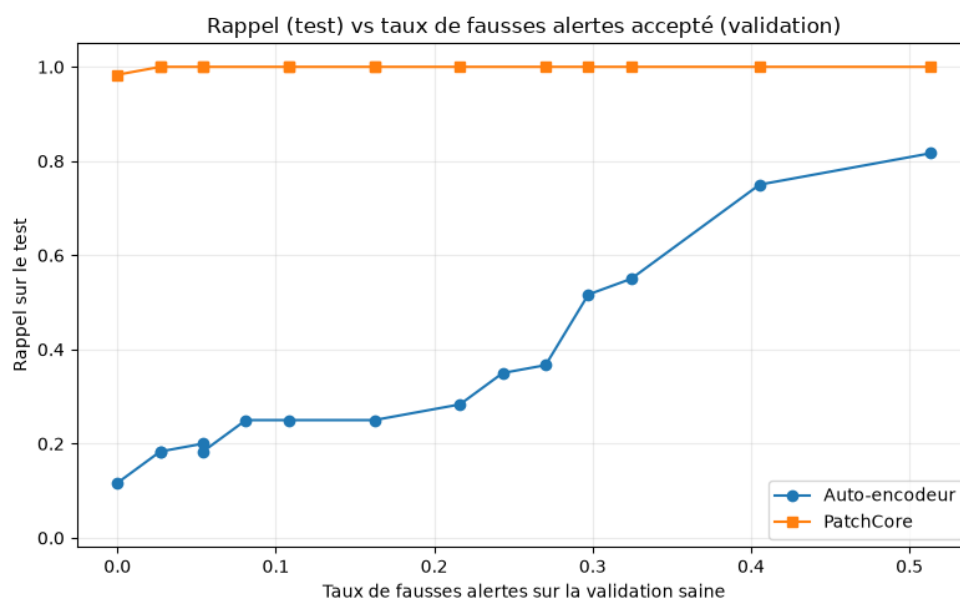
Méthode

Réentraînement de l'auto-encodeur de référence et de PatchCore sur `wood`, puis balayage de **15 politiques de seuil** par modèle (percentiles 50 à 99, moyenne + k-écart-type pour k de 0,5 à 3), calibrées uniquement sur la validation saine. La politique retenue l'est par une règle fixée **avant** de lire les résultats de test — maximiser le rappel, puis minimiser le taux de fausses alertes en cas d'égalité — jamais en cherchant après coup la meilleure ligne du tableau (biais déjà signalé en section 4).

Justification

Les seuils explorés en section 4 restaient proches de la politique par défaut. La décision métier impose d'aller beaucoup plus loin : jusqu'où peut-on pousser le rappel, pour chaque modèle, et à quel coût en fausses alertes ?

Résultats

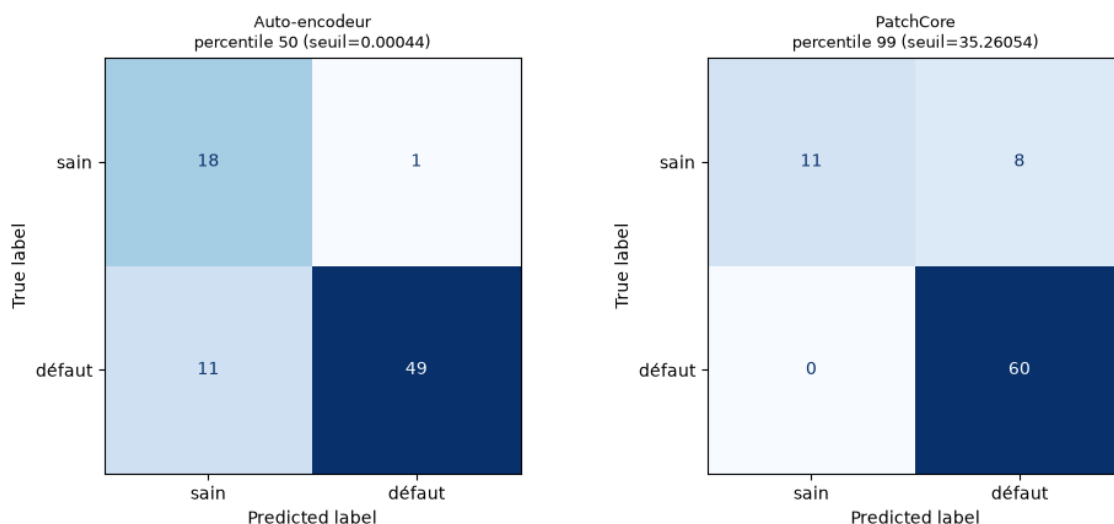


Rappel sur le test en fonction du taux de fausses alertes accepté sur la validation — auto-encodeur vs PatchCore.

L'auto-encodeur plafonne à un rappel de 0,817, quel que soit le seuil. Même au seuil le plus permissif testé (percentile 50 — la moitié des pièces saines de validation dépassent déjà ce seuil), 11 défauts sur 60 restent non détectés : les erreurs de reconstruction des défauts les plus discrets (liquid) sont structurellement trop proches de celles des images saines pour qu'un seuil, quel qu'il soit, les sépare — une limite du **score** lui-même, pas de la politique de seuil.

PatchCore atteint un rappel de 1,000 sur presque toute la plage testée — du percentile 50 au percentile 99, et de moyenne + $0,5\sigma$ à moyenne + 2σ . Le taux de fausses alertes nécessaire pour ce rappel parfait est beaucoup plus faible que pour l'auto-encodeur : dès percentile 99 (2,7 % de fausses alertes sur la validation), le rappel est déjà de 1,000.

Modèle	Politique retenue	FA (validation)	Précision (test)	Rappel (test)
Auto-encodeur	percentile 50 (la plus permissive testée)	51,4 %	0,98	0,817
PatchCore	percentile 99	2,7 %	0,882	1,000



Matrices de confusion des politiques retenues — auto-encodeur (percentile 50) vs PatchCore (percentile 99).

Décision finale. Modèle et seuil retenus pour la production : **PatchCore, seuil calibré au 99e centile de la validation saine.** Rappel de 100 % (0 défaut manqué) pour 2,7 % de fausses alertes sur la validation (8 fausses alertes sur 19 pièces saines de test) — contre un rappel plafonné à 81,7 % pour l'auto-encodeur, même au prix d'une fausse alerte sur une pièce saine sur deux. Ce choix est désormais **implémenté en code de production** dans `src/indusense/vision/patchcore.py` (classe `PatchCoreDetector` : `fit`, `calibrate_threshold`, `score`, `predict`, `anomaly_map`, `save/load`), avec le 99e centile comme seuil par défaut.

Limites à surveiller avant un déploiement réel. Le test ne compte que 79 images : un taux de fausses alertes de 2,7 % repose sur une seule fausse alerte de plus ou de moins pouvant le faire varier sensiblement. La règle de choix ignore délibérément tout compromis de précision au-delà de ce qui est nécessaire pour atteindre le rappel maximal — cohérent avec la décision métier, mais à ne pas généraliser telle quelle à un contexte où le coût d'une fausse alerte serait significatif. Seule la catégorie wood est couverte par ce choix de seuil ; la comparaison de la section 6.4 suggère que PatchCore serait au moins aussi justifié sur metal_nut. Enfin, l'extraction de features du backbone convolutif de PatchCore sur GPU n'est pas garantie bit-exacte : la valeur numérique du seuil peut varier légèrement d'un réentraînement à l'autre (contrairement à l'auto-encodeur, vérifié déterministe) — recalibrer systématiquement après chaque `fit()`, jamais réutiliser un seuil d'un entraînement précédent.

8. Synthèse transversale

Enseignements qui traversent l'ensemble de ces expériences, au-delà de chaque résultat pris isolément :

- **Classement $\neq$ décision.** Un excellent ROC-AUC/PR-AUC (qualité du classement des scores) ne garantit jamais une bonne précision/rappel au seuil choisi (qualité de la décision) : constaté sur la comparaison frameworks (§1), la comparaison de seuils (§4, §7) et le TP officiel (§5) — trois contextes différents, la même leçon.
- **Détection $\neq$ localisation.** L'AUROC image (une image contient-elle un défaut ?) et l'AUROC pixel (où est le défaut ?) sont deux tâches différentes qui peuvent diverger fortement (§5, §6.2) : un bon classement global n'implique pas une bonne carte de chaleur.
- **Le coût de « première fois » sur GPU n'est pas une constante.** Le surcoût de première époque pour une forme de tenseur inédite (résolution §2, noyau de convolution §3) est un coût d'auto-tuning MIOpen **ponctuel, mis en cache sur disque** par forme et par machine — spectaculaire à froid ($52\times$ / $33,6\times$), négligeable à chaud ($\sim 2\text{-}4\times$).
- **PyTorch reproductible, TensorFlow/Keras CPU non reproductible.** Vérifié empiriquement en ré-exécutant les mêmes notebooks à plusieurs jours d'intervalle (§1) : à prendre en compte pour toute comparaison chiffrée entre frameworks.
- **Le piège du ratio de compression est réel, pas seulement théorique.** Vérifié numériquement avant tout entraînement (§5.1, ratio 0,75) puis confirmé à l'extrême sur metal_nut (§6.3, AUROC pire que le hasard) : l'hypothèse de l'auto-encodeur par reconstruction dépend fortement de la nature des données (texture homogène vs objet réfléchissant).
- **L'auto-encodeur par reconstruction est un baseline pédagogique, pas un état de l'art.** Confirmé quantitativement (§6.4) : PatchCore le surclasse sur les deux catégories testées, et résout précisément le cas où l'auto-encodeur échoue le plus violemment.
- **Un rappel plafonné signale une limite du score, pas du seuil.** Sur wood, l'auto-encodeur ne dépasse jamais 0,817 de rappel, même au seuil le plus permissif imaginable (§7) : quand des défauts ont un score aussi bas que les images saines, aucun seuil ne les sépare — seul un meilleur score (donc un autre modèle) résout le problème.
- **Une décision métier se traduit par une règle de seuil fixée à l'avance, pas par un choix a posteriori.** Le balayage de 15 politiques par modèle (§7) n'a de valeur que parce que la règle de sélection (rappel maximal, puis fausses alertes minimales) a été énoncée avant de lire les résultats de test — sans quoi le choix final serait sur-ajusté aux 79 images de test.

Limites assumées. Une seule catégorie « objet » (metal_nut) et une seule configuration PatchCore (wide_resnet50_2, couches layer2/layer3, coreset 10 %) ont été testées ; les comparaisons de politiques de seuil (§4, §7) portent sur un test de 79 images (petit échantillon) ; généraliser ces conclusions à d'autres objets, d'autres backbones ou des jeux de test plus larges resterait à vérifier.

9. Risques et robustesse pour un déploiement réel

Les sections précédentes évaluent le classement des scores (ROC-AUC/PR-AUC) et la décision au seuil retenu (précision/rappel) sur le jeu MVTec AD, dans des conditions homogènes avec l'entraînement. Elles ne couvrent pas les risques propres à un déploiement industriel réel, détaillés dans le cours « **Risques éthiques et robustesse** » (voir *docs/02-sprint-2/04-revisions/16-risques-biais-drift-robustesse-ethique-vision.md*).

- **Biais de dataset.** Les images d'entraînement et de validation (§§0, 5, 6) proviennent d'une capture homogène (même éclairage, même cadrage que MVTec AD). Une caméra de production avec un éclairage, un angle ou une ligne différents produirait le même type de biais que celui décrit dans le cours : bonnes performances mesurées ici, dégradées en production — un point non testé faute d'images de production réelles.
- **Data drift / concept drift.** Ni l'auto-encodeur ni PatchCore n'incluent de mécanisme de surveillance continue des performances ni de réentraînement périodique. Le seuil PatchCore retenu en §7 (99e centile, valeur 35,26) est figé à la date de calibration : sans monitoring des scores de production dans le temps, une dérive progressive (nouvelle caméra, usure, nouveau lot) ne serait détectée qu'après une remontée des défauts manqués.
- **Robustesse aux données hors distribution.** L'échec de l'auto-encodeur sur *metal_nut* (§6.3, AUROC image = 0,2556, sous le hasard) illustre concrètement une architecture sans garde-fou effectif face à une variation photométrique non vue à l'entraînement (reflets spéculaires), alors que PatchCore — comparaison locale à une mémoire de features pré-entraînées — reste robuste (§6.4). Le cours souligne que l'auto-encodeur par reconstruction dispose en théorie d'un garde-fou similaire face à une donnée franchement hors distribution (mauvaise reconstruction = alerte) ; l'échec mesuré ici en montre la limite pratique dès qu'une variation n'est pas clairement séparable du signal de défaut.
- **Attaques adversariales et qualité d'acquisition.** Ni la robustesse à des perturbations adverses (quelques pixels modifiés, invisibles à l'œil nu) ni la sensibilité à une caméra mal calibrée (mise au point, poussière) ne sont testées ici (jeu de test MVTec AD propre et contrôlé) ; à vérifier avant tout déploiement.
- **Confidentialité / RGPD.** Sans objet sur MVTec AD (pièces industrielles, aucune personne identifiable), mais à réévaluer sur un jeu d'images de production réel selon leur contenu.
- **Garde-fous de production.** La politique retenue en §7 (PatchCore, seuil au 99e centile) applique une décision entièrement automatique à seuil binaire. Le cours recommande une zone d'incertitude à vérification humaine (*human-in-the-loop*) pour les scores proches du seuil, une documentation formelle des cas exclus du périmètre (carte de modèle) et un comportement de repli défini en cas d'indisponibilité du modèle — aucun des trois n'est mis en place dans ce projet.

Ces points ne remettent pas en cause les choix techniques retenus dans ce document (PatchCore préféré à l'auto-encodeur, seuil au 99e centile) : ils rappellent qu'un bon classement et un seuil bien calibré sur un jeu de test ne suffisent pas, seuls, à garantir un déploiement sûr et robuste, et documentent ce qui resterait à traiter avant une mise en production réelle.